02. 가상 머신 (VM) 구현 상세 분석

2.1 VM 구조체 정의

2.1.1 핵심 데이터 구조

```
pub struct VM {
stack: [Mersenne31; 4], // 스택 (4개 요소)
sp: usize, // 스택 포인터 (현재 미사용)
instructions: Vec<Instructions>, // 실행할 명령어 목록
ip: usize, // 명령어 포인터 (Instruction Pointer)
trace: Vec<VMState>, // 실행 추적 기록
}
```

각 필드 설명:

- `stack` : Mersenne31 필드 원소 4개로 구성된 고정 크기 스택
- `sp` : 스택 포인터 (구현에서는 실제로 사용되지 않음)
- `instructions` : 실행할 프로그램 (명령어 벡터)
- `ip` : 현재 실행 중인 명령어의 인덱스
- `trace` : 각 명령어 실행 후의 VM 상태를 기록

2.1.2 VMState 구조체

```
pub struct VMState {
stack: [Mersenne31; 4], // 명령어 실행 후 스택 상태
instruction: Instructions, // 실행된 명령어
extra_data: Mersenne31, // 추가 데이터 (나눗셈의 나머지 등)
}
```

VMState의 역할:

- 각 명령어 실행 후의 완전한 상태 스냅샷
- 증명 생성을 위한 추적 데이터의 기본 단위
- `extra_data` 는 나눗셈 연산의 나머지 값을 저장

2.2 명령어 정의

2.2.1 Instructions 열거형

```
pub enum Instructions {
Push(Mersenne31), // 값을 스택에 푸시
Add, // 덧셈
Sub, // 뺄셈
Mul, // 곱셈
Div, // 나눗셈
}
```

2.3 VM 초기화

2.3.1 VM::new 함수

```
pub fn new(insruction: Vec<Instructions>) -> Self {
Self {
stack: [Mersenne31::ZERO; 4],
sp: 0,
instructions: insruction,
ip: 0,
trace: vec![],
}
}
```

초기화 과정:

1. 스택을 모두 0으로 초기화
2. 명령어 포인터를 0으로 설정
3. 빈 추적 벡터 생성

2.4 프로그램 실행

2.4.1 VM::run 함수 상세 분석

```
pub fn run(&mut self) -> Result<(), String> {
while self.ip < self.instructions.len() {
```

```
let  instruction = self.instructions[self.ip].clone();
self.ip += 1;


type BinaryOp = fn(Mersenne31, Mersenne31) -> Mersenne31;
let  mut  extra_data: Mersenne31 = Mersenne31::ZERO;
match &instruction {
Instructions::Push(val) => {
// Push 구현
}
Instructions::Add => {
self.perform_operation(|a, b| a + b, None::<BinaryOp>)?;
}
Instructions::Sub => {
self.perform_operation(|a, b| a - b, None::<BinaryOp>)?;
}
Instructions::Mul => {
self.perform_operation(|a, b| a * b, None::<BinaryOp>)?;
}
Instructions::Div => {
extra_data = self.perform_operation(
|a, b| a / b,
Some(|a, b| a - (a / b) * b)
)?;
```

```
}
}

self.trace.push(VMState {
stack: self.stack,
instruction: instruction,
extra_data: extra_data,
});
}
Ok(())
}
```

실행 흐름:

1. 명령어 포인터가 프로그램 끝에 도달할 때까지 반복
2. 현재 명령어를 가져오고 ip를 증가
3. 명령어 타입에 따라 적절한 연산 수행
4. 실행 후 상태를 trace에 기록

2.5 명령어별 구현 상세

2.5.1 Push 명령어

```
Instructions::Push(val) => {
for i in (1..self.stack.len()).rev() {
```

```
self.stack[i] = self.stack[i - 1];
}
self.stack[0] = *val;
}
```

동작 방식:

```
실행 전: [a, b, c, d]
Push(x) 실행
실행 후: [x, a, b, c]
```

1. 스택의 모든 요소를 한 칸씩 뒤로 이동
2. 새 값을 스택의 맨 앞(인덱스 0)에 배치
3. 맨 뒤의 값(d)은 사라짐

예시:

```
// 초기 스택: [0, 0, 0, 0]
Push(10) // [10, 0, 0, 0]
Push(20) // [20, 10, 0, 0]
Push(30) // [30, 20, 10, 0]
```

2.5.2 이항 연산 (Add, Sub, Mul, Div)

모든 이항 연산은 `perform_operation` 함수를 통해 처리됩니다.

```
fn perform_operation<F, G>(
&mut self,
operation: F,
sub_operation: Option<G>,
) -> Result<(Mersenne31), String>
where
F: Fn(Mersenne31, Mersenne31) -> Mersenne31,
G: Fn(Mersenne31, Mersenne31) -> Mersenne31,
{
let b = self.stack[1]; // 두 번째 피연산자
let a = self.stack[0]; // 첫 번째 피연산자
let result = operation(a, b);
let mut extra_data: Mersenne31 = Mersenne31::ZERO;
match sub_operation {
Some(f) => {
extra_data = f(a, b);
}
None => {}
}
// 스택 재배치
for i in 2..4 {
self.stack[i - 1] = self.stack[i];
```

```
}
self.stack[0] = result;
self.stack[3] = Mersenne31::ZERO;
Ok((extra_data))
}
```

동작 방식:

```
실행 전: [a, b, c, d]
연산 실행 (예: Add)
실행 후: [a+b, c, d, 0]
```

1. 스택의 상위 두 값(a, b)을 가져옴
2. 연산 수행: `result = operation(a, b)`
3. 스택 재배치: [c, d]를 앞으로 이동
4. 결과를 맨 앞에 배치
5. 맨 뒤를 0으로 채움

2.5.3 각 연산의 구체적 예시

Add 연산

```
// 스택: [20, 10, 0, 0]
Add 실행
// a = 20, b = 10
// result = 20 + 10 = 30
```

```
// 결과: [30, 0, 0, 0]
```

Sub 연산

```
// 스택: [20, 10, 0, 0]
Sub 실행
// a = 20, b = 10
// result = 20 - 10 = 10
// 결과: [10, 0, 0, 0]
```

Mul 연산

```
// 스택: [20, 10, 0, 0]
Mul 실행
// a = 20, b = 10
// result = 20 * 10 = 200
// 결과: [200, 0, 0, 0]
```

Div 연산 (특별한 경우)

```
// 스택: [23, 10, 0, 0]
Div 실행
// a = 23, b = 10
// result = 23 / 10 = 2 (정수 나눗셈)
```

```
// extra_data = 23 - (23/10)*10 = 23 - 20 = 3 (나머지)

// 결과: [2, 0, 0, 0], extra_data = 3
```

Div의 sub_operation:

- `|a, b| a - (a / b) * b`: 나머지 계산
- 이 나머지 값은 AIR 제약 조건에서 나눗셈의 정확성을 검증하는 데 사용됨

2.6 실행 추적 생성

2.6.1 VM::get_trace 함수

이 함수는 VM의 내부 추적을 증명 시스템이 사용할 수 있는 형태로 변환합니다.

```
pub fn get_trace(&self) -> Vec<[Mersenne31; 11]> {

let mut final_trace: Vec<[Mersenne31; 11]> = vec![[Mersenne31::ZERO; 11]];

for i in self.trace.iter() {

match i.instruction {

Instructions::Push(val) => {

final_trace.push([

i.stack[0], i.stack[1], i.stack[2], i.stack[3], // 스택 4개

val, // Push 값

Mersenne31::ONE, // Push 플래그

Mersenne31::ZERO, // Add 플래그

Mersenne31::ZERO, // Sub 플래그

Mersenne31::ZERO, // Mul 플래그
```

```
Mersenne31::ZERO, // Div 플래그

i.extra_data, // 추가 데이터

]);

}

// 다른 명령어들도 유사한 패턴...

}

}

final_trace

}
```

2.6.2 추적 테이블 구조 (11개 컬럼)

| 인덱스 | 이름 | 설명 |

|--------|------|------|

| 0 | stack[0] | 스택 최상위 값 |

| 1 | stack[1] | 스택 두 번째 값 |

| 2 | stack[2] | 스택 세 번째 값 |

| 3 | stack[3] | 스택 네 번째 값 |

| 4 | push_val | Push 명령어의 값 |

| 5 | is_push | Push 명령어 플래그 (1 또는 0) |

| 6 | is_add | Add 명령어 플래그 |

| 7 | is_sub | Sub 명령어 플래그 |

| 8 | is_mul | Mul 명령어 플래그 |

| 9 | is_div | Div 명령어 플래그 |

| 10 | extra_data | 추가 데이터 (나머지 등) |

원-핫 인코딩 (One-Hot Encoding):

- 각 행에서 정확히 하나의 명령어 플래그만 1
- 나머지 플래그는 모두 0
- 이를 통해 AIR 제약 조건에서 명령어 타입을 구분

2.6.3 추적 예시

프로그램:

```
Push(10)

Push(20)

Add
```

생성되는 추적 테이블:

```
Row | s[0] | s[1] | s[2] | s[3] | push_val | is_push | is_add | is_sub | is_mul

----|------|------|------|------|----------|---------|--------|--------|--------

0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0

1 | 10 | 0 | 0 | 0 | 10 | 1 | 0 | 0 | 0 | 0 | 0

2 | 20 | 10 | 0 | 0 | 20 | 1 | 0 | 0 | 0 | 0 | 0

3 | 30 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0
```

2.7 테스트 케이스 분석

2.7.1 Add 연산 테스트

```
#[test]
fn check_add_operation() {
let program = vec![
Instructions::Push(Mersenne31::from_canonical_u32(10)),
Instructions::Push(Mersenne31::from_canonical_u32(20)),
Instructions::Add,
];

let mut vm = VM::new(program);
vm.run().unwrap();

assert_eq!(
vm.stack,
[
Mersenne31::from_canonical_u32(30),
Mersenne31::ZERO,
Mersenne31::ZERO,
Mersenne31::ZERO
]
);
}
```

실행 과정:

1. Push(10): [10, 0, 0, 0]
2. Push(20): [20, 10, 0, 0]
3. Add: [30, 0, 0, 0] ✓

2.7.2 복잡한 연산 체인

```
// 테스트 코드의 복잡한 예시
Push(10) // [10, 0, 0, 0]
Push(20) // [20, 10, 0, 0]
Add // [30, 0, 0, 0]
Push(40) // [40, 30, 0, 0]
Sub // [10, 0, 0, 0] (40 - 30)
Push(2) // [2, 10, 0, 0]
Mul // [20, 0, 0, 0] (2 * 10)
Push(23) // [23, 20, 0, 0]
Div // [1, 0, 0, 0] (23 / 20 = 1, 나머지 3)
```

2.8 VM의 핵심 설계 원칙

2.8.1 결정론적 실행

- 같은 프로그램은 항상 같은 결과 생성
- 난수나 외부 입력 없음
- 완전히 재현 가능한 실행

2.8.2 완전한 상태 추적

- 모든 중간 상태 기록
- 증명 생성에 필요한 모든 정보 포함
- 검증자가 실행을 재구성할 수 있음

2.8.3 유한체 연산

- 모든 연산은 Mersenne31 필드에서 수행
- 오버플로우 없음 (모듈로 연산)
- 암호학적으로 안전한 연산

이 VM 구현은 간단하지만 zkVM의 핵심 개념을 모두 포함하고 있습니다. 다음 문서에서는 이 추적 데이터를 사용하여 어떻게 증명을 생성하는지 살펴보겠습니다.